



Powered by  
**Arizona State University®**

# LOGICA DE PROGRAMACION

---

Trabajo Autónomo N.º 1

**ANALIZAR E IMPLEMENTAR LAS FUNCIONALIDADES Y LA  
ARQUITECTURA DEL SOFTWARE, APLICANDO LOS CONOCIMIENTOS  
ADQUIRIDOS DURANTE LAS SEMANAS DE CLASE.**

---

**DOCENTE:**  
**CABEZAS ERAZO DARIO SEBASTIAN**

**ESTUDIANTE:**  
**LINCANGO MOROCHO MAYERLY ABIGAIL**

**Repositorio GitHub:** <https://github.com/Abigailuchi/juego-ahorcado>

**FECHA DE ENTREGA:**  
**23-06-2026**



## 1. INTRODUCCION

---

Para esta materia de Lógica de Programación nos pidieron desarrollar un proyecto en el que pudiéramos aplicar todo lo que hemos aprendido durante el semestre, como las estructuras condicionales y los bucles. En el Autónimo 1 ya habíamos elegido un problema y habíamos hecho un diagrama de flujo para representar cómo se resolvería, y ahora en este Autónimo 2 el objetivo es convertir ese diagrama en un programa real, escrito en código. El proyecto que yo elegí desde el Autónimo 1 es el juego del Ahorcado, que es un juego bastante conocido en el que hay que ir adivinando las letras de una palabra oculta antes de quedarse sin intentos. Me pareció un buen proyecto porque tiene varias decisiones lógicas que se pueden representar fácilmente con condicionales (si la letra está o no está en la palabra) y también necesita un bucle, porque el juego se repite mientras el jugador sigue intentando adivinar.

En este documento voy a explicar cómo armé mi repositorio en GitHub, cómo fui pasando la lógica del diagrama de flujo a código en Python, y cómo en el Paso 2 mejoré la documentación de mi código para que se entienda mejor. El objetivo principal de este trabajo es demostrar que el diagrama que hice antes realmente funciona cuando se convierte en un programa de verdad.

### 1.1 Antecedentes

Este trabajo parte directamente de lo que se hizo en el Autónimo 1, donde tuvimos que elegir un problema cualquiera y representarlo mediante un diagrama de flujo, sin todavía escribir código. En esa entrega yo elegí como problema el juego del Ahorcado, un juego conocido en el que un jugador debe adivinar las letras de una palabra oculta antes de quedarse sin intentos disponibles. En ese momento diseñé el diagrama completo, definiendo cómo el sistema elegiría la palabra, cómo se revisaría cada letra ingresada por el jugador, y cómo terminaría el juego, ya sea ganando o perdiendo. Ese diagrama de flujo del Autónimo 1 es el antecedente directo de todo lo que se desarrolla en este documento, ya que el Autónimo 2 no parte de un problema nuevo, sino que retoma esa misma lógica para llevarla a código real, en el lenguaje de programación Python, y posteriormente mejorar su documentación interna.

## 2. OBJETIVOS

---



## 2.1 Objetivo general

Convertir el diagrama de flujo del juego del Ahorcado, elaborado en el Autónimo 1, en un programa funcional escrito en Python, aplicando estructuras condicionales y repetitivas, y documentando el código de manera clara para que cualquier persona pueda entenderlo.

## 2.2 Objetivos específicos

- Configurar un repositorio en GitHub y un entorno de desarrollo adecuado para programar el proyecto.
- Trasladar cada parte del diagrama de flujo a una instrucción concreta de código en Python, usando condicionales (if, else) y un bucle (while).
- Comprobar que el programa funcione correctamente, tanto cuando el jugador gana como cuando pierde.
- Mejorar la documentación del código en el Paso 2, agregando comentarios que expliquen qué recibe y qué devuelve cada función.

## 3. Marco Teórico

---

Antes de explicar lo que hice, dejo claros algunos conceptos que usé, para que se entienda mejor por qué tomé ciertas decisiones al programar.

### 3.1 Diagrama de flujo

Es un dibujo con figuras (óvalos, rectángulos, rombos) que muestra paso a paso cómo funciona un proceso, antes de programarlo. Los rombos representan una decisión, como "¿la letra está en la palabra?", y según la respuesta, el dibujo se va por un lado o por otro.

### 3.2 Condicionales (if, else)

Sirven para que el programa haga una cosa u otra dependiendo de si se cumple una condición. En Python se escriben con if ("si") y else ("si no"). En mi juego, uso un if para revisar si la letra que el jugador escribió está dentro de la palabra secreta o no, y dentro de eso, otro if para revisar si ya ganó o si ya perdió.

### 3.3 Bucles (while)



Repiten un mismo pedazo de código varias veces. Usé un bucle while, que se sigue repitiendo mientras el jugador no haya ganado ni perdido todavía, y se detiene con la instrucción break apenas se cumple una de esas dos condiciones.

### 3.4 Python y el tipo de dato set

Python es el lenguaje que usamos en la materia, bastante fácil de leer. Para este proyecto usé un tipo de dato llamado set (conjunto), que guarda varios valores sin que se repitan, y lo usé para ir guardando las letras que el jugador ya adivinó correctamente.

### 3.5 GitHub y los commits

GitHub es una página donde se sube el código de un proyecto y se guarda un historial de los cambios. Cada vez que se guarda un avance se le llama commit. Lo usamos para que el profesor pueda ver cómo fue avanzando el proyecto, no solo el resultado final.

## 4. METODOLOGÍA

---

A continuación explico, paso por paso, cómo fui desarrollando el proyecto, siguiendo lo que se pedía tanto en el Paso 1 como en el Paso 2 del Autónomo 2.

### 4.1 Crear el repositorio y preparar el entorno

Lo primero que hice fue crear un repositorio nuevo en GitHub, llamado juego-ahorcado, dejándolo público. Después instalé Visual Studio Code, que es el editor donde escribo el código, y Git, que es lo que permite subir los cambios a GitHub. Cloné mi repositorio a la computadora con el comando git clone, y configuré mi nombre de usuario y mi correo con git config, porque sin eso no se puede hacer ningún commit.

### 4.2 Subir el diagrama de flujo

Como ya tenía el diagrama de flujo hecho desde el Autónomo 1, lo guardé como imagen y lo subí dentro de mi repositorio, para que quede ahí junto con el código.

### 4.3 Código en Python

El código lo dividí en tres funciones para que quedara más ordenado: una para elegir la palabra al azar, otra para mostrar el progreso del jugador con guiones, y la función principal, que junta todo y controla el juego completo usando el bucle while y los condicionales if y else.



La función que elige la palabra no recibe ningún dato, simplemente escoge una de una lista de cinco palabras que yo puse (Python, Teclado, Programa, Variable y Función) usando `random.choice`. La función que muestra el progreso sí recibe dos datos: la palabra secreta, y el conjunto de letras que el jugador ya adivinó. Lo que hace es recorrer la palabra letra por letra, y si esa letra ya está en el conjunto de adivinadas, la muestra normal; si no, pone un guion bajo en su lugar.

Y en la función principal, primero elijo la palabra, creo un conjunto vacío para las letras adivinadas, y le doy cinco intentos al jugador. Después entra al bucle `while`, que le pide una letra al jugador una y otra vez. Si la letra está en la palabra, la agrego al conjunto de adivinadas y reviso si con eso ya se completó toda la palabra (si es así, gana). Si la letra no está, le resto un intento y reviso si ya se quedó sin intentos (si es así, pierde). Cuando pasa cualquiera de las dos cosas, se rompe el bucle con `break` y el juego termina.

#### 4.4 Mejorar la documentación del código (Paso 2)

En esta segunda parte, el profesor explicó en clase que no había que cambiar la lógica del programa, sino más bien mejorar los comentarios dentro del código, para que cualquier persona que lo lea pueda entender qué hace cada función, aunque no sepa programar. Entonces volví a mi código y le agregué comentarios más completos, explicando en cada función qué información recibe y qué es lo que devuelve al final.

#### 4.5 Probar que el programa funcione

Por último, ejecuté el programa varias veces desde la terminal de Visual Studio Code, jugando partidas completas para asegurarme de que tanto el caso de ganar como el de perder funcionaran bien, y de que todo coincidiera con lo que había planeado en el diagrama de flujo.

## 5. RESULTADOS

---

En esta parte muestro las evidencias de lo que logré hacer, tanto en el Paso 1 como en el Paso 2.

#### 5.1 Diagrama de flujo del juego

En la Figura 1 se ve el diagrama de flujo que diseñé en el Autófono 1 y que usé como base para programar. El juego empieza cuando el sistema elige una palabra al azar y la muestra oculta con guiones. Después el jugador va escribiendo letras, y se revisa si esa letra está en la palabra: si sí está, se revela y se revisa si ya se completó toda la palabra (ahí se gana); si no está, se le quita un intento al jugador y se revisa si ya se le acabaron (ahí se pierde). Esto se repite, representado con las flechas punteadas que regresan hacia arriba, hasta que pasa una de esas dos cosas.



Figura 1. Diagrama de flujo que hice para el juego del Ahorcado.

## 5.2 Repositorio en GitHub

En la Figura 2 se ve cómo quedó mi repositorio llamado juego-ahorcado en GitHub. Ahí se nota que tiene los tres archivos que se pedían: el README, el diagrama de flujo en imagen, y el archivo ahorcado.py con el código. También se ve el número de commits que fui haciendo, lo cual demuestra que fui guardando avances poco a poco y no subí todo de una sola vez al final.

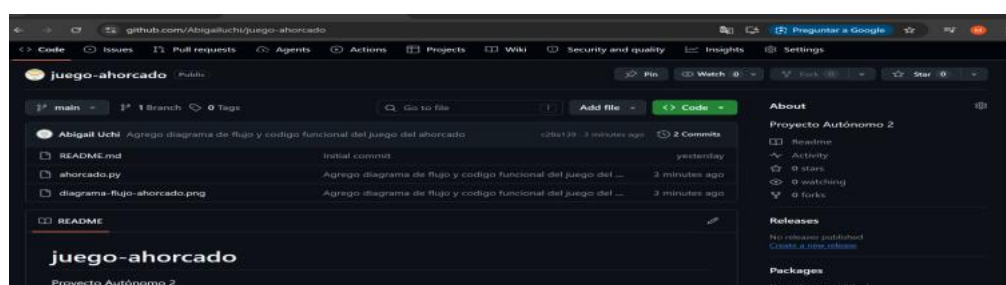


Figura 2. Mi repositorio en GitHub con los archivos y los commits.

## 5.3 Las funciones de mi programa

Para organizar mejor el código, lo dividí en tres funciones, que explico en la Tabla 1



| Función                         | ¿Qué hace?                                                                                                                   |
|---------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| <code>elegir_palabra()</code>   | Escoge una palabra al azar de una lista de 5 palabras que yo puse. No recibe ningún dato.                                    |
| <code>mostrar_progreso()</code> | Recibe la palabra y las letras adivinadas, y devuelve la palabra con guiones donde aún falta adivinar.                       |
| <code>jugar()</code>            | Es la función principal: pide letras, revisa si gano o pierdo con <code>if</code> y <code>while</code> , y termina el juego. |

Tabla 1. Las tres funciones de mi programa.

#### 5.4 El programa funcionando

Para comprobar que todo funcionaba, ejecuté el programa en la terminal, como se ve en la Figura 3. En esa prueba la palabra que tocó fue "PYTHON", y fui escribiendo las letras hasta completarla. El programa fue revelando cada letra correcta, y al final salió el mensaje de "Ganaste!" y después "Fin del juego".

```
ahorcado.py
22 def jugar():
55
56     print("Fin del juego")
57
58
59 if __name__ == "__main__":
60     jugar()

PROBLEMS OUTPUT TERMINAL
PS C:\Users\PC\Desktop\juego-ahorcado> python ahorcado.py
Ingresar una letra: y
P Y _ H O _
Ingresar una letra: t
P Y T H O _
Ingresar una letra: n
P Y T H O N
Ganaste!
Fin del juego
```

#### 5.5 Cómo se relaciona el diagrama con el código

En la Tabla 2 pongo cómo cada parte de mi diagrama de flujo (Figura 1) se ve reflejada en una parte específica de mi código. Hacer esta comparación me ayudó a confirmar que no me faltó programar ninguna parte de lo que había diseñado antes.

| Parte del diagrama            | Parte del código                                                   |
|-------------------------------|--------------------------------------------------------------------|
| El sistema elige una palabra  | <code>elegir_palabra()</code>                                      |
| El usuario ingresa una letra  | <code>input()</code> dentro del <code>while</code>                 |
| ¿La letra está en la palabra? | <code>if letra in palabra:</code>                                  |
| Revelar la letra              | <code>letras_adivinadas.add(letra)</code>                          |
| Restar un intento             | <code>intentos -= 1</code>                                         |
| ¿Quedan intentos?             | <code>if intentos &lt;= 0:</code>                                  |
| ¿Se completó la palabra?      | <code>if set(palabra) == letras_adivinadas:</code>                 |
| Ganaste / Perdiste            | <code>print("Ganaste!")</code> o <code>print("Perdiste...")</code> |
| Fin del juego                 | <code>print("Fin del juego")</code>                                |



## 5.6 Código completo

Aquí dejo el código completo de mi archivo ahorcado.py, ya con los comentarios que mejoré para el Paso 2.

```
"""
Juego del Ahorcado -

Este programa lleva a código real el algoritmo que fue diseñado
previamente en el diagrama de flujo (Autonomo 1).

Flujo del programa (igual al diagrama):
Inicio -> el sistema elige una palabra -> se muestra oculta con guiones ->
el usuario ingresa una letra -> se decide si la letra está en la palabra ->
si está: se revela la letra y se revisa si la palabra quedó completa (Ganaste) ->
si no está: se resta un intento y se revisa si quedan intentos (Perdiste) ->
Fin del juego.
"""

import random

def elegir_palabra():
    """
    Sistema elige una palabra.

    No recibe ningún parámetro.

    Devuelve (return) un texto (str) en mayúsculas, elegido al azar
    de una lista fija de palabras relacionadas con programación.
    """
    palabras = ["PYTHON", "TECLADO", "PROGRAMA", "VARIABLE", "FUNCION"]

    return random.choice(palabras)

def mostrar_progreso(palabra, letras_adivinadas):
    """
    Construye el texto que el jugador ve en pantalla.
    """
```





#### Parametros:

palabra (str): la palabra secreta que el sistema eligio.

letras\_adivinadas (set): conjunto con las letras que el jugador ya adivino correctamente hasta el momento.

Devuelve (return) un texto (str) donde cada letra adivinada se muestra normal, y cada letra que falta se muestra como un guion bajo.

Por ejemplo, si la palabra es "PYTHON" y solo se adivino la "P", esta funcion devuelve: "P \_ \_ \_ \_"

#### Courier New

```
# El usuario ingresa una letra

letra = input("Ingresa una letra: ").upper()

# Decision (estructura condicional): la letra esta en la palabra?
if letra in palabra:

    # Caso "Si" del diagrama: Revelar la letra correcta
    letras_adivinadas.add(letra)

    print(mostrar_progreso(palabra, letras_adivinadas))

    # Nueva decision: se completo la palabra?
    if set(palabra) == letras_adivinadas:

        print("Ganaste!")

        break # Termina el bucle: Fin del juego
else:

    # Caso "No" del diagrama: Restar un intento
    intentos -= 1

    print(f"Letra incorrecta. Te quedan {intentos} intentos.")

    # Nueva decision: quedan intentos?
    if intentos <= 0:

        print(f"Perdiste! La palabra era: {palabra}")

    break # Termina el bucle: Fin del juego
```

#### Courier New

```
# Este bloque hace que el programa solo se ejecute si abrimos
# directamente este archivo (y no si lo importamos desde otro archivo).
if __name__ == "__main__":

    jugar()
```



## 6. Conclusiones

---

Haciendo este trabajo me di cuenta de que el diagrama de flujo que había hecho en el Autónomo 1 sí me sirvió de mucho, porque ya tenía pensada toda la lógica antes de empezar a programar, y eso hizo que escribir el código fuera más fácil de lo que pensaba, porque ya sabía qué tenía que hacer cada parte. También aprendí a usar mejor los condicionales y los bucles, ya que en este proyecto no eran solo un ejemplo de clase, sino que tenía que pensar yo misma en qué momento usar un `if` y en qué momento usar un `while` para que el juego funcionara de verdad. Otra cosa que aprendí fue la importancia de comentar bien el código. Al principio yo entendía mi propio código porque lo acababa de escribir, pero cuando tuve que explicarlo para el Paso 2, me di cuenta de que si alguien más lo leyera sin comentarios, le costaría entender qué hace cada función. Por eso creo que el consejo del profesor de que el código documentado es más fácil de repetir y de mejorar tiene mucho sentido. Por último, también fue la primera vez que usé GitHub de esta forma, subiendo avances poco a poco con commits, y me pareció útil porque así uno no pierde el progreso y se puede ver cómo fue cambiando el proyecto. En general, este trabajo me ayudó a entender mejor todo lo que hemos visto en la materia, y me deja más preparada para los proyectos que vienen.